



Lesson 12

03.08.2026

```
public class Ex1 {
    static int a = 1111;

    static {
        System.out.println("static");
        a = a-- - --a;
    }

    {
        System.out.println("non static");
        a = a++ + ++a;
    }

    public static void main(String[] args) {
        System.out.println(a);
    }
}
```

```
public class Ex2 {  
    public static void main(String[] args) {  
        Integer i1 = 128;  
        Integer i2 = 128;  
        System.out.println(i1 == i2);  
  
        Integer i3 = 127;  
        Integer i4 = 127;  
        System.out.println(i3 == i4);  
    }  
}
```



```
public class Ex3 {  
    public static void show() {  
        System.out.println("Static method called");  
    }  
  
    public static void main(String[] args) {  
        Ex3 obj = null;  
        obj.show();  
    }  
}
```

```
public class Ex4 {  
    static int method1(int i) {  
        return method2(i *= 11);  
    }  
    static int method2(int i) {  
        return method3(i /= 11);  
    }  
    static int method3(int i) {  
        return method4(i -= 11);  
    }  
    static int method4(int i) {  
        return i += 11;  
    }  
    public static void main(String[] args) {  
        System.out.println(method1(11));  
    }  
}
```

1

Тема: Типи в Java



Що таке ссылочний тип і актуальний тип?

Ссылочный тип (Reference Type)

- Це тип змінної (посилання).
- Визначає, які методи можна викликати на етапі компіляції.
- Компілятор бачить тільки цей тип.

Актуальний тип (Actual Type)

- Це реальний тип об'єкта в пам'яті.
- Визначає, який код буде виконано під час роботи програми (runtime).
- JVM дивиться на реальний об'єкт у пам'яті.



Простими словами:

Ссылочный тип – це "що ми бачимо",
а актуальний тип – це "що є насправді".



2

Приклад: Notification і Push

```
class Notification {
    public void send() {
        System.out.println("send notif...");
    }
}

class Push extends Notification {
    @Override
    public void send() {
        System.out.println("send push...");
        super.send(); // викликає Notification.send()
    }

    public void print() {
        System.out.println("print push");
    }
}
```



Push наслідує Notification.

Push перевизначає метод `send()`
і додає свій метод `print()`.

3

Як це виглядає в пам'яті?

Стек (Stack)
пам'ять для змінних

Змінна `not`
(тип: `Notification`)

посилання
0x1234

Куча (Heap)
пам'ять для об'єктів

Адреса: 0x1234
Об'єкт типу `Push`



Методи об'єкта `Push`:

- `send()` // override
- `print()`
- `super.send()`
(`Notification.send()`)

Змінна `not` зберігає адресу (0x1234) об'єкта в кучі.
Актуальний тип об'єкта – `Push`.

4

Сумісність типів

Чому `Notification` сумісний з `Push`?

`Notification`

(базовий клас)

+ `send()`

extends →

`Push`

(нащадок)

+ `send()` // override
+ `print()`
+ `super.send()`
(`Notification.send()`)

`Push` є нащадком (subclass) класу `Notification`.

Тому об'єкт типу `Push` можна посилатися змінною
типу `Notification` (upcasting):

`Notification not = new Push();` // ✅ можна

Але не можна навпаки (downcasting без приведення):

`Push push = new Notification();` // ❌ помилка компіляції



5

Що можна викликати?

Чо вирішує компілятор, а що – JVM?

`Notification not = new Push();`

На етапі компіляції
(компілятор дивиться на тип змінної)

Тип змінної: `Notification`

Доступні методи:

- ✅ `send()`
- ❌ `print()` // недоступний

Під час виконання
(JVM дивиться на реальний об'єкт)

Реальний об'єкт: `Push`

Виконується метод:

```
not.send()
↓
Push.send() (override)
↓
super.send()
↓
Notification.send()
```

6

Підсумок

Що це?	Ссылочный тип (Reference Type)	Актуальний тип (Actual Type)
Що визначає?	Які методи можна викликати	Який код реально виконається
Коли визначається?	На етапі компіляції	Під час виконання (runtime)
Де зберігається?	У стеку (тип змінної)	В об'єкті в кучі (heap)

Приклади викликів

```
Notification not = new Push();
not.send(); // ✅ компілюється
not.print(); // ❌ не компілюється
```

```
Push push = new Push();
push.send(); // ✅
push.print(); // ✅
```

★ Головне правило поліморфізму в Java

Тип змінної (посилання) визначає, що можна викликати.
Тип об'єкта визначає, який метод буде виконано.



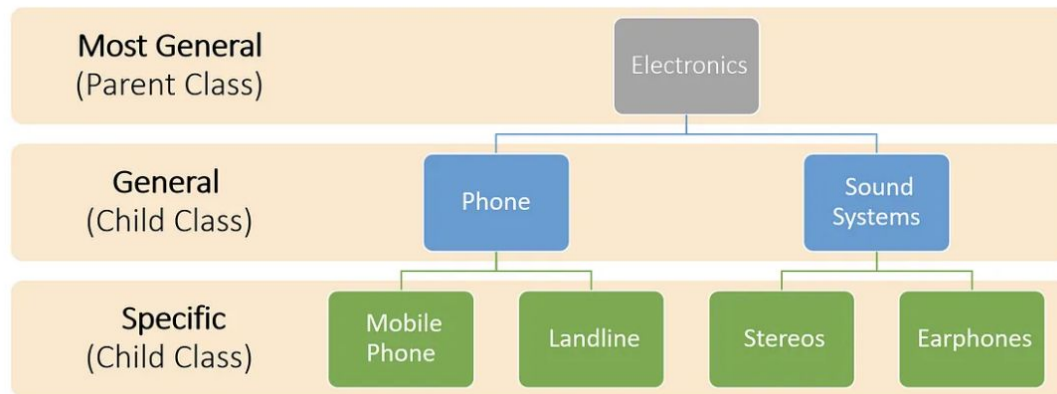
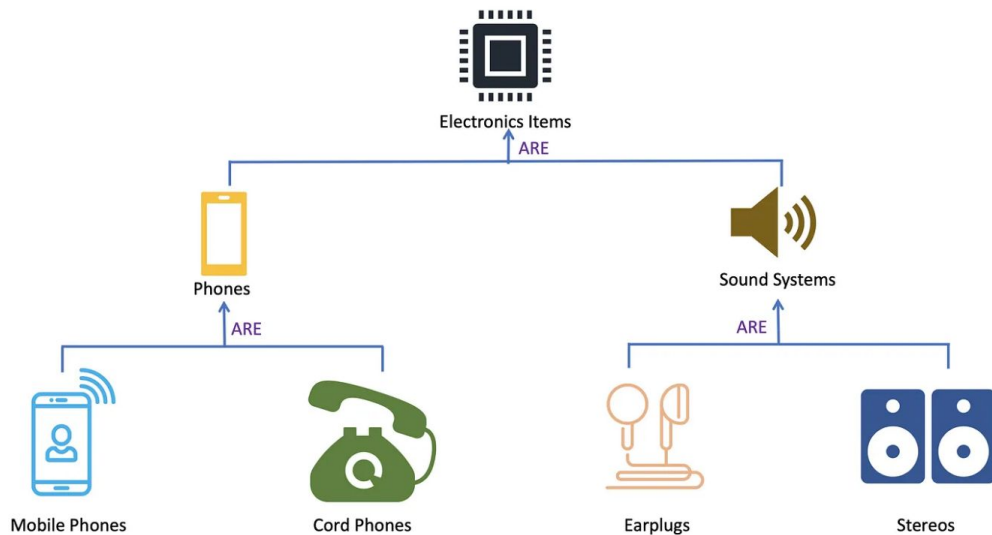


Успадкування

Успадкування — це механізм ООП, який дозволяє створювати нові класи на основі вже існуючих. Новий клас (підклас) отримує всі публічні та захищені поля і методи батьківського класу (суперкласу).

Основні особливості успадкування:

- Дозволяє уникнути дублювання коду.
- Забезпечує повторне використання функціональності.
- Дозволяє створювати ієрархію класів.





Ціль: повторне використання коду, розширення функціональності, поліморфізм.

НАСЛІДУВАННЯ У JAVA

Наслідування – це механізм, який дозволяє одному класу (нащадку) успадковувати поля та методи іншого класу (батьківського).

НАВИЩО?

- Повторне використання коду
- Логічна ієрархія класів
- Поліморфізм
- Легше супроводжувати та розширювати



РЕЗУЛЬТАТ:

Ієрархія класів, легша підтримка, розширення функціональності та поліморфізм.

1. СИНТАКСИС

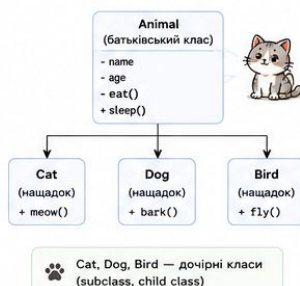
Клас-нащадок використовує ключове слово **extends** для успадкування від батьківського класу.

```
class Parent {  
    // поля та методи  
}  
  
class Child extends Parent {  
    // нові поля та методи  
}
```



Java не підтримує множинне спадкування класів (через **extends**), але можна успадковувати через інтерфейси.

2. ІЄРАРХІЯ КЛАСІВ: ПРИКЛАД



3. ПРИКЛАД КОДУ

```
class Animal {  
    String name;  
    void eat() {  
        System.out.println("Тварина їсть");  
    }  
    void sleep() {  
        System.out.println("Тварина спить");  
    }  
}  
  
class Cat extends Animal {  
    void meow() {  
        System.out.println("Мее!");  
    }  
    @Override  
    void eat() { // перевизначення методу  
        System.out.println("Кіт їсть корн");  
    }  
}
```

4. ВИКОРИСТАННЯ

```
Cat cat = new Cat(); // об'єкт типу Cat  
cat.name = "Мурчик";  
cat.eat(); // викликає перевизначений  
cat.sleep(); // метод з Animal  
cat.meow(); // унікальний для Cat  
  
Animal animal = cat; // upcasting (підняття типу)  
animal.eat(); // викликає eat() з Cat  
animal.sleep(); // викликає sleep() з Animal  
// animal.meow(); // ПОМИЛКА: метод не видно  
  
Cat cat2 = (Cat) animal; // downcasting (приведення типу)  
cat2.meow(); // тепер доступний метод Cat
```

⚠️ Увага! Downcasting потребує явного приведення типу. Можна отримати **ClassCastException** у runtime.

5. ЩО УСПАДКОВУЄТЬСЯ?



УСПАДКОВУЄТЬСЯ:

- ✓ Поля (окрім private)
- ✓ Методи (окрім private)
- ✓ захищені (protected) члени
- ✓ Публічні (public) члени

НЕ УСПАДКОВУЄТЬСЯ:

- ✗ Конструктори
- ✗ Приватні поля та методи
- ✗ Статичні члени (вони приховуються)

6. ПЕРЕВИЗНАЧЕННЯ МЕТОДІВ (OVERRIDING)

Дочірній клас може надати свою реалізацію методу батьківського класу.

```
class Animal {  
    void speak() {  
        System.out.println("Тварина видає звук");  
    }  
}  
  
class Dog extends Animal {  
    @Override  
    void speak() {  
        System.out.println("Гав-гав!");  
    }  
}
```

Правила перевизначення:

- ✓ Такий самий підпис методу (ім'я, параметри)
- ✓ Не можна зменшувати область видимості
- ✓ Можна повертати коваріантний тип
- ✓ Можна викидати менше або ті самі винятки



7. ТИПИ НАСЛІДУВАННЯ

Одинарне (single)

Клас має лише одного батька.



Ієрархічне (hierarchical)

Один батько – багато нащадків.



Багаторівневе (multilevel)

Нащадок стає батьком для наступного класу.



Множинне (multiple)

Java НЕ підтримує множинне наслідування класів. Використовуйте інтерфейси.



8. КЛЮЧОВЕ СЛОВО super

Використовується для звернення до членів батьківського класу.

- ✓ Виклик конструктора батьківського класу: **super();**
- ✓ Виклик методу батьківського класу: **super.method();**
- ✓ Виклик поля батьківського класу: **super.field;**

```
class Cat extends Animal {  
    Cat(String name) {  
        super.name = name; // доступ до поля батька  
    }  
    void eat() {  
        super.eat(); // виклик методу батька  
        System.out.println("Кіт їсть рибу");  
    }  
}
```



9. ПОЛІМОРФІЗМ ЗАВДЯКИ НАСЛІДУВАННЮ

Один інтерфейс (тип батька) – багато реалізацій (типів нащадків).

```
Animal a1 = new Cat();  
Animal a2 = new Dog();  
Animal a3 = new Bird();  
  
a1.eat(); // викликає реалізацію Cat  
a2.eat(); // викликає реалізацію Dog  
a3.eat(); // викликає реалізацію Bird
```

💡 Який би об'єкт не був створений – викликається своя реалізація методу!



10. ПРАВИЛО



Використовуйте наслідування, коли є відношення IS-A (є-видом). Наприклад: Кіт є Твариною.



Не використовуйте наслідування, якщо класи просто пов'язані, а не є різними видами одного. У такому випадку краще використовувати композицію.



ПРИКЛАД РЕАЛЬНОГО СЦЕНАРІЮ

У банківській системі:

Account (базовий клас) → SavingsAccount, CreditAccount, BusinessAccount
Кожен тип рахунку має спільні властивості та поведінку (наприклад, balance, deposit, withdraw), але може мати і свої особливості.



КОРИСНО ЗНАТИ

final клас не може мати нащадків.
final метод не може бути перевизначений.
Object – батько всіх класів у Java.





Інкапсуляція

Інкапсуляція — це один із ключових принципів об'єктно-орієнтованого програмування (ООП), який полягає у приховуванні деталей реалізації класу та забезпеченні доступу до даних лише через визначені методи.

Основні ідеї інкапсуляції:

1. **Приховування реалізації** — внутрішні деталі класу не доступні для зовнішнього використання.
2. **Контроль доступу** — дані можна змінювати лише через спеціально визначені методи.
3. **Захист від некоректних змін** — встановлюючи обмеження доступу, ми запобігаємо неправильному використанню даних.



ІНКАПСУЛЯЦІЯ У JAVA

Інкапсуляція – це приховування внутрішнього стану об'єкта та надання доступу до нього лише через публічні методи (гетери та сетери).



НАВИЩО?

- Захист даних від некоректного доступу
- Контроль над змінами стану об'єкта
- Гнучкість та безпечніше супроводження коду

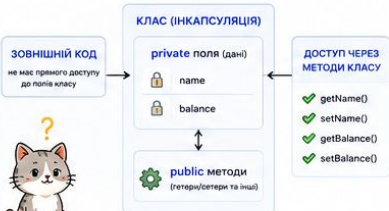


РЕЗУЛЬТАТ:

Безпека, цілісність даних, легше підтримувати та змінювати реалізацію.

1. ШО ТАКЕ ІНКАПСУЛЯЦІЯ?

Ми приховуємо поля класу від прямого доступу ззовні (робимо їх private) і дозволяємо працювати з ними лише через публічні методи.



2. ПРИКЛАД КЛАСУ

```
public class BankAccount {  
    // 1) Приватні поля (стан об'єкта)  
    private String owner;  
    private double balance;  
  
    // 2) Гетери (отримання значень)  
    public String getOwner() {  
        return owner;  
    }  
    public double getBalance() {  
        return balance;  
    }  
  
    // 3) Сетери (зміна значень з перевіркою)  
    public void setOwner(String owner) {  
        if (owner == null || owner.isBlank()) {  
            throw new IllegalArgumentException("Owner cannot be empty");  
        }  
        this.owner = owner;  
    }  
    public void setBalance(double balance) {  
        if (balance < 0) {  
            throw new IllegalArgumentException("Balance cannot be negative");  
        }  
        this.balance = balance;  
    }  
  
    // 4) Інші поведінкові методи  
    public void deposit(double amount) {  
        if (amount <= 0) throw new IllegalArgumentException();  
        balance += amount;  
    }  
    public void withdraw(double amount) {  
        if (amount <= 0 || amount > balance)  
            throw new IllegalArgumentException("Insufficient funds");  
        balance -= amount;  
    }  
}
```



3. ЯК ВИКОРИСТОВУВАТИ

```
BankAccount account = new BankAccount();  
account.setOwner("Олександр");  
account.setBalance(1000.0); // через метод  
  
System.out.println(account.getOwner()); // Олександр  
System.out.println(account.getBalance()); // 1000.0  
  
account.deposit(500);  
account.withdraw(200);  
System.out.println(account.getBalance()); // 1300.0
```



НЕ МОЖНА (прямий доступ заборонено)

```
account.owner = "Іван"; // ПОМИЛКА компіляції  
account.balance = -100; // ПОМИЛКА компіляції
```



Результат:

Зовнішній код не може напряму змінити поля на некоректні значення. Усі зміни проходять перевірку в методах класу.



4. ПЕРЕВАГИ ІНКАПСУЛЯЦІЇ

- Захист даних**
Дані не можна змінити напряму на некоректні.
- Контроль та валідація**
У методах можемо перевіряти аргументи, обмеження, логіку.
- Гнучкість реалізації**
Можемо змінити внутрішню реалізацію класу, не ламаючи зовнішній код.
- Менша зв'язаність**
Код стає зрозумілішим, легше тестувати і підтримувати.



5. РІВНІ ДОСТУПУ В JAVA

Модифікатор	Видимість	Де доступний	Приклад
private	тільки всередині класу	лише в цьому класі	private int age;
(без модифікатора) (default)	в межах пакета	класи того ж пакета	int balance;
protected	в пакеті та в інших пакетах	пакет + підкласи в інших пакетах	protected void log();
public	скрізь	буди-де	public String getName();

6. ПРИКЛАД: БЕЗ ІНКАПСУЛЯЦІЇ vs з ІНКАПСУЛЯЦІЄЮ

БЕЗ ІНКАПСУЛЯЦІЇ (погано)

```
class UserBad {  
    public String name;  
    public int age;  
}  
  
UserBad u = new UserBad();  
u.name = null; // можна зіпсувати дані  
u.age = -10; // некоректні значення
```



з ІНКАПСУЛЯЦІЄЮ (добре)

```
class UserGood {  
    private String name;  
    private int age;  
    public void setAge(int age) {  
        if (age < 0) throw new IllegalArgumentException();  
        this.age = age;  
    }  
    public int getAge() { return age; }  
}  
  
// Дані захищені та валідовані
```



7. ПРАВИЛО

Робить поля класу private, а доступ надавайте через публічні методи.
Це основа інкапсуляції в Java!



8. ПІДСУМОК

- Інкапсуляція = private поля + публічні методи.
- Приховуємо стан об'єкта від зовнішнього коду.
- Керуємо доступом і перевіряємо дані.
- Робить код безпечнішим, гнучкішим та підтримуваним.



ПРИКЛАД РЕАЛЬНОГО СЦЕНАРІЮ

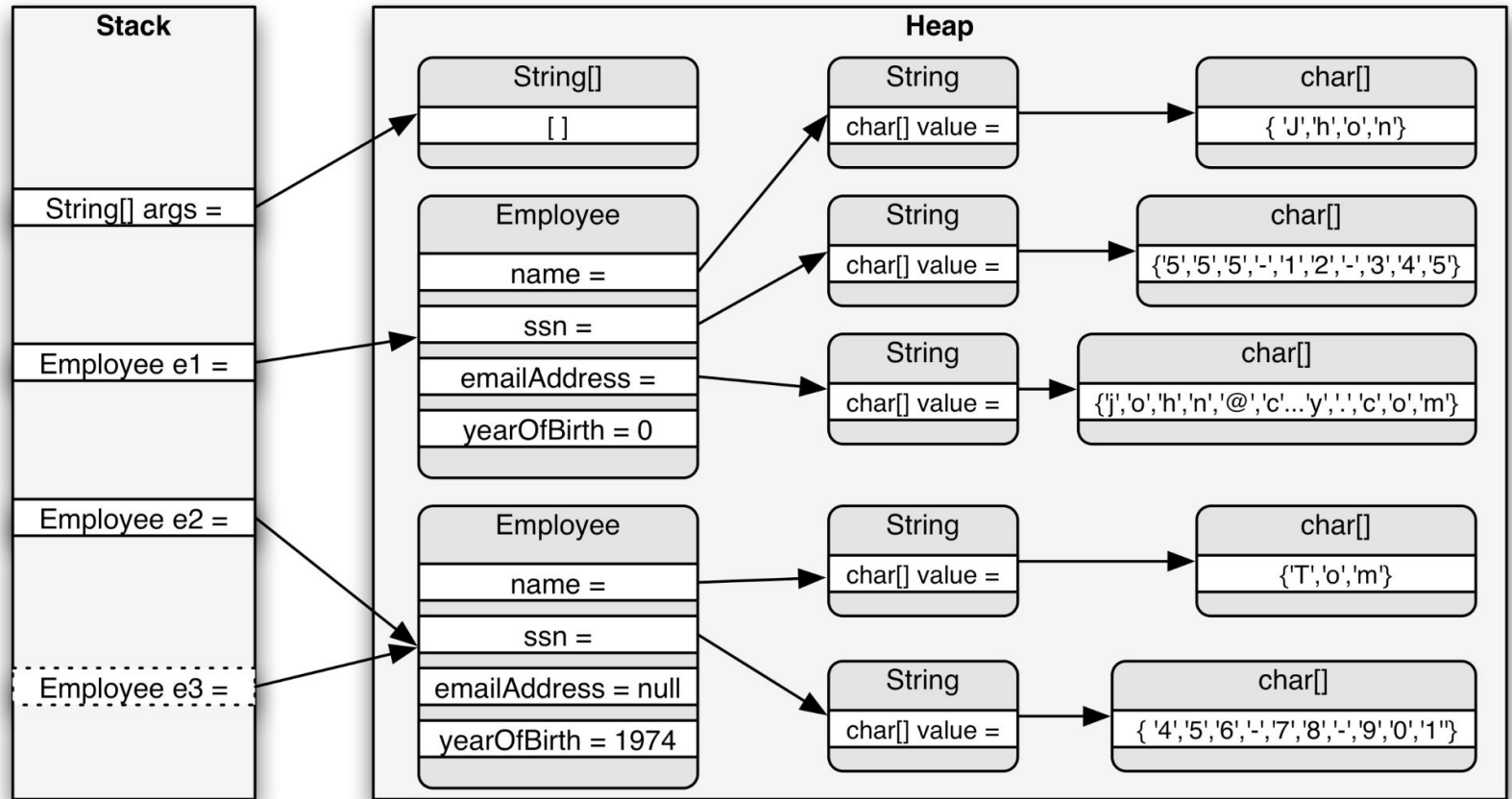
У банківській системі баланс рахунку не можна змінювати напряму. Його можна поповнити або зняти тільки через методи deposit() і withdraw(), які перевіряють суми та доступність коштів.

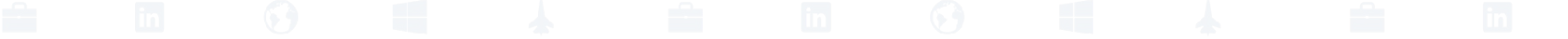


ВАЖЛИВО ЗНАТИ

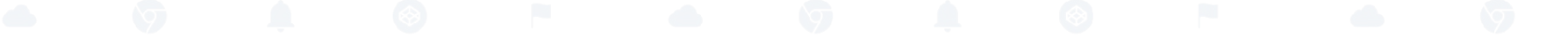
Інкапсуляція працює разом з іншими принципами ООП: спадкуванням, поліморфізмом та абстракцією.

Клас – це капсула, що зберігає дані та знає, як з ними працювати.





Java variables do not contain the actual objects, they contain *references* to the objects.

- The actual objects are stored in an area of memory known as the *heap*.
 - Local variables referencing those objects are stored on the stack.
 - More than one variable can hold a reference to the same object.
- 
- 

Клас `Object` визначає метод `clone()`, що створює копію об'єкта. Якщо ви хочете, щоб екземпляр вашого класу можна було клонувати, необхідно перевизначити цей метод та реалізувати інтерфейс `Cloneable`. Інтерфейс `Cloneable` – це інтерфейс-маркер, він не містить ні методів, ні змінних. Інтерфейси-маркер просто визначають поведінку класів.

`Object.clone()` викидає виняток `CloneNotSupportedException` при спробі клонувати об'єкт, що не реалізує інтерфейс `Cloneable`. Метод `clone()` у батьківському класі `Object` є `protected`, тому бажано перевизначити його як `public`. Реалізація за умовчанням методу `Object.clone()` виконує неповне/поверхнєве (shallow) копіювання

```
public class Student implements Cloneable {  
  
    @Override  
    protected Object clone() throws CloneNotSupportedException {  
        return super.clone();  
    }  
}
```



КЛОНУВАННЯ ОБ'ЄКТІВ У JAVA

Клонування – це створення копії об'єкта, яка має такі ж значення полів, але є окремим незалежним об'єктом у пам'яті.

НАВІЩО?

- Створення резервної копії об'єкта
- Безпечне модифікування копії
- Реалізація шаблонів (Prototype тощо)



РЕЗУЛЬТАТ:

Отримуємо новий об'єкт з однаковим станом, але іншим місцем у пам'яті.

1. ШО ТАКЕ КЛОНУВАННЯ?



obj1 і obj2 – різні об'єкти (різні адреси в пам'яті), але мають однакові значення полів. Зміни у копії не впливають на оригінал (для глибокого копіювання).

2. ЯК ПРАЦЮЄ CLONE В JAVA?

- Клас повинен реалізувати інтерфейс Cloneable
- Переопределити метод Object clone()
- Метод clone() створює копію об'єкта

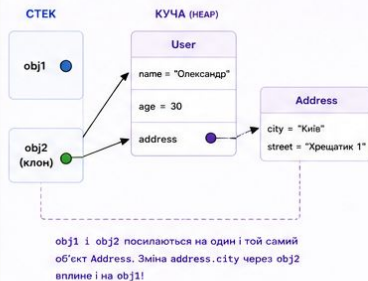
```
public class User implements Cloneable {
    private String name;
    private int age;
    private Address address; // складове поле

    @Override
    protected Object clone() throws CloneNotSupportedException {
        return super.clone(); // поверхове клонування
    }
}
```

Метод Object.clone() за замовчуванням виконує поверхове клонування (shallow copy).

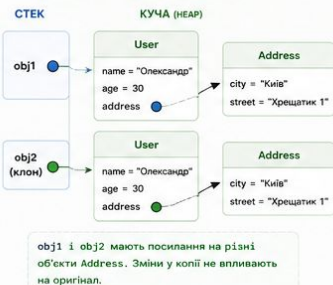
3. ПАМ'ЯТЬ: ПОВЕРХОВЕ (SHALLOW) vs ГЛИБОКЕ (DEEP) КЛОНУВАННЯ

ПОВЕРХОВЕ КЛОНУВАННЯ (shallow copy)



obj1 і obj2 посилаються на один і той самий об'єкт Address. Зміна address.city через obj2 вплине і на obj1!

ГЛИБОКЕ КЛОНУВАННЯ (deep copy)



obj1 і obj2 мають посилання на різні об'єкти Address. Зміни у копії не впливають на оригінал.

4. ПРИКЛАД КОДУ

```
class Address implements Cloneable {
    String city;
    String street;

    Address(String city, String street) {
        this.city = city;
        this.street = street;
    }

    @Override
    protected Object clone() throws CloneNotSupportedException {
        return super.clone();
    }
}

class User implements Cloneable {
    String name;
    int age;
    Address address;

    User(String name, int age, Address address) {
        this.name = name;
        this.age = age;
        this.address = address;
    }

    @Override
    protected Object clone() throws CloneNotSupportedException {
        return super.clone(); // поверхове копіювання
    }

    // Глибоке копіювання
    public User deepClone() throws CloneNotSupportedException {
        User cloned = (User) super.clone();
        cloned.address = (Address) this.address.clone();
        return cloned;
    }
}
```

```
public class Main {
    public static void main(String[] args) throws Exception {
        Address addr = new Address("Київ", "Хрещатик 1");
        User original = new User("Олександр", 30, addr);

        // Поверхове клонування
        User shallow = (User) original.clone();

        // Глибоке клонування
        User deep = original.deepClone();

        // Змінимо місто через shallow
        shallow.address.city = "Львів";

        System.out.println("Оригінал: " + original.address.city);
        System.out.println("Shallow: " + shallow.address.city);
        System.out.println("Deep: " + deep.address.city);
    }
}
```

РЕЗУЛЬТАТ:

Оригінал: Київ
Shallow: Львів // зміна вплинула на оригінал!
Deep: Київ // глибока копія не залежить



5. ВАЖЛИВО ЗНАТИ

- clone() у Object має модифікатор protected, тому викликається всередині класу або його нащадків.
- Якщо клас містить посилання на інші об'єкти, потрібно реалізувати глибоке клонування.
- Інтерфейс Cloneable не містить методів – це лише маркер.
- Якщо не переопределити clone(), буде створено поверхову копію.
- Масиви вже підтримують клонування:

```
int[] arr = {1, 2, 3};
int[] copy = arr.clone();
```



6. АЛЬТЕРНАТИВИ КЛОНУВАННЮ



Конструктор копіювання

Створіть конструктор, який приймає об'єкт і копіює поля.



Builder / Factory / Prototype

Використовуйте патерн Prototype для створення копій.



Серіалізація

Серіалізуйте об'єкт у байти та десеріалізуйте назад (повне глибоке клонування).



КОЛИ ВИКОРИСТОВУВАТИ?

- Коли потрібно швидко отримати копію об'єкта.
- Коли об'єкт складний і конструктор копіювання незручний.
- У шаблонах проектування (Prototype).



ПІДСУМОК

Клонування дозволяє створювати копії об'єктів. За замовчуванням – це поверхове копіювання. Для повної незалежності потрібно реалізувати глибоке копіювання.

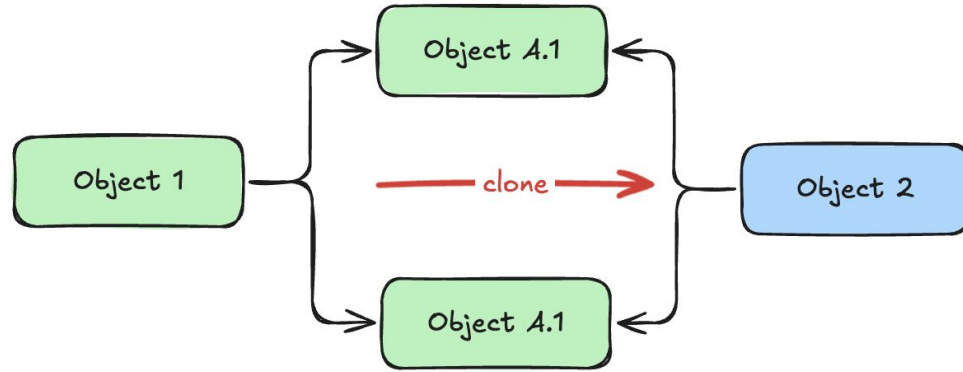


ЛЕГЕНДА

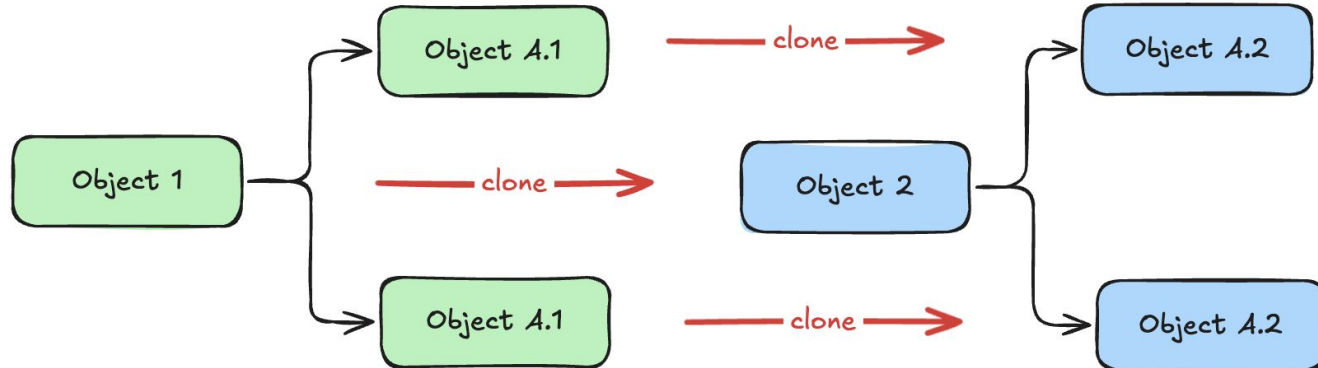
- посилання на об'єкт у купі
- > внутрішнє посилання всередині об'єкта



Shallow Cloning



Deep Cloning





this vs super

Чим **this** і **super** схожі

- І **this**, і **super** - це нестатичні змінні, відповідно їх не можна використовувати у статичному контексті, а це означає, що їх не можна використовувати у методі `main`. Це призведе до помилки під час компіляції "на нестатичну змінну цього не можна посилатися зі статичного контексту". те саме станеться, якщо в методі `main` скористатися ключовим словом `super`.
- І **this**, і **super** можуть використовуватися всередині конструкторів для виклику інших конструкторів по ланцюжку, нпр., **this()** і **super()** викликають конструктор без аргументів спадкового та батьківського класів відповідно.
- Всередині конструктора `this` і `super` повинні стояти вище за всіх інших виразів, на початку, інакше компілятор видасть повідомлення про помилку. З чого випливає, що в одному конструкторі не може бути одночасно і `this()`, та `super()`.

Відмінності у super та this

- Змінна `this` посилається на поточний екземпляр класу, в якому вона використовується, тоді як `super` - на екземпляр батьківського класу.
- Кожен конструктор за відсутності явних викликів інших конструкторів неявно викликає за допомогою `super()` конструктор без аргументів батьківського класу, при цьому у вас завжди залишається можливість явно викликати будь-який інший конструктор з допомогою або `this()`, або `super()`.



this і super – ключові посилання в Java, які допомагають працювати з поточним об'єктом та батьківським класом.

THIS vs SUPER у Java

НАВИЩО?

- Уникати конфліктів імен
- Викликати потрібний конструктор або метод
- Писати зрозумілий та безпечний код

РЕЗУЛЬТАТ:

Чистий, гнучкий код і коректна робота з успадкуванням.

1. THIS – посилання на поточний об'єкт

this вказує на поточний екземпляр класу.

	Поле класу	Розрізняє поле об'єкта від параметра з такою ж назвою.
	Метод класу	Викликає інший метод цього ж класу.
	Конструктор класу	Викликає інший конструктор цього ж класу.
	Передача поточного об'єкта	Передає поточний об'єкт як аргумент у метод.

ПРИКЛАДИ

```
class Person {
    private String name;

    // 1) Поле класу
    Person(String name) {
        this.name = name; // this.name = поле об'єкта
    }

    // 2) Метод класу
    void setName(String name) {
        this.name = name;
    }

    // 3) Виклик іншого конструктора
    Person() {
        this("Unknown");
    }

    // 4) Передача об'єкта
    void printInfo() {
        print(this);
    }

    void print(Person p) {
        System.out.println("name: " + p.name);
    }
}
```



2. SUPER – посилання на батьківський клас

super використовується в нащадку для роботи з членами батьківського класу.

	Поле батьківського класу	Звертається до поля батьківського класу, якщо воно приховано (hidden).
	Метод батьківського класу	Викликає перевизначений метод батьківського класу.
	Конструктор батьківського класу	Викликає конструктор батьківського класу. Має бути першим у конструкторі нащадка.

ПРИКЛАДИ

```
class Animal {
    String name = "Тварина";
    Animal(String name) {
        this.name = name;
    }
    void speak() {
        System.out.println("Тварина видає звук");
    }
}

class Dog extends Animal {
    String name = "Собака"; // приховує поле батька
    Dog(String name) {
        super(name); // 3) виклик конструктора батька
    }

    void show() {
        System.out.println("this.name = " + this.name); // Собака
        System.out.println("super.name = " + super.name); // Тварина
    }

    @Override
    void speak() {
        super.speak(); // 2) виклик методу батька
        System.out.println("faw-rau!");
    }
}
```



3. ЯК ЦЕ ПРАЦЮЄ В ПАМ'ЯТІ

У методі об'єкта нащадка можна використовувати:

- **this** – посилання на поточний об'єкт (екземпляр Dog)
- **super** – посилання на частину цього об'єкта, що належить батьківському класу (Animal)



ПРИКЛАД ВИКЛИКУ

```
Dog dog = new Dog("Барбос");
dog.show();
```

Вивід:

```
this.name = Собака
super.name = Барбос
```



4. ОСОБЛИВОСТІ ТА ПРАВИЛА

- ✓ **this** можна використовувати в будь-якому методі або конструкторі класу.
- ✓ **this()** / **this(...)** – виклик іншого конструктора цього ж класу. Має бути першим у конструкторі.
- ✓ **super()** / **super(...)** – виклик конструктора батьківського класу. Має бути першим у конструкторі нащадка.
- ✓ **this** і **super** не можна використовувати в статичних (static) методах або полях.



5. ПОШИРЕНІ ПОМИЛКИ

- ✗ Забути **this** – відбулося присвоєння параметра самому собі.
- ✗ Виклик **super()** не є першим у конструкторі.
- ✗ Спроба використати **this** або **super** у static контексті.



6. ПОРІВНЯННЯ THIS vs SUPER

Що	this	super
На що вказує	на поточний об'єкт	на батьківський клас (частину об'єкта)
Доступ	до членів поточного класу	до членів батьківського класу
Конструктор	this() – інший конструктор цього ж класу	super() – конструктор батьківського класу
Виклик метода	інший метод поточного класу	перевизначений метод батьківського класу
Випадки використання	усунення конфліктів імен, виклик своїх методів/конструкторів, передача себе як аргумент	доступ до прихованих членів батька, повторне використання логіки батька, ініціалізація батьківської частини об'єкта



КЛЮЧОВЕ СЛОВО

Використовуйте **this** і **super** правильно – і ваш код буде зрозумілим, безпечним та готовим до масштабування!



ПРИКЛАД РЕАЛЬНОГО СЦЕНАРІЮ

У банківській системі клас **Account** (базовий) містить загальні дані. Клас **SavingsAccount** (нащадок) використовує **super** для виклику логіки батька та **this** для роботи зі своїми даними.

